some cases they prove to work quite well in practice, on real-world data.

How is this possible? That's because the theory of NP-hardness is about *worst-case performance*, and for some problems there are a minority of edge cases that turn out to be hard to solve; but for practical applications we are often interested in *average-case hardness*, aka how difficult it is to solve a problem, on average, on the instances of the problem that we see in real scenarios.

There are many heuristics that have been developed for graph algorithms, for example to solve the traveling salesman problem[4], or finding a clique on a graph[5]. Not all the heuristics are the same, of course: usually they are a compromise between performance and accuracy, and for some of these algorithms it's possible to prove bounds on their precision, for example proving that the solution they output will be within a certain margin from the optimal solution.

So, what kind of heuristic could we use to find a good (or at least decent) graph embedding?

Now, I'd like you to keep in mind what we said a few lines above: there can be many different approximate algorithms for each problem, and not all of them are equally as good.

We'll start easy, with a simple heuristic, that's far from ideal: it will serve a purpose, let us understand the problem scenario, and then we can (and will) iterate over it to improve its performance.

Before we start with our first attempt, I encourage you to think about this, before moving on, and try to develop your idea: who knows, you might have a breakthrough and come up with a new solution!